Sample data

```
{
  "_id": ObjectId,
  "orderId": String,
  "customerName": String,
  "orderDate": Date,
  "status": String,        // e.g., "Pending", "Shipped", "Delivered"
  "items": [
    { "productName": String, "quantity": Number, "price": Number }
  ],
  "totalAmount": Number
}
```

You can create at least **15–20 dummy records** manually or use **faker.js / any data generator**.

Part 1: Aggregation

Tasks

1. **Find total revenue generated** (sum of `totalAmount`).
2. **Find total number of orders by status** (`Pending`, `Shipped`, `Delivered`).
3. **Find the top 3 customers who spent the most** (sort by `totalAmount`).
4. **Get the average order amount per customer**.
5. **Find products that were sold more than 10 times (total quantity)**.
6. **List monthly revenue (group by month-year)** for the last 6 months.
7. **Find all customers who placed more than 2 orders**.
8. **Extract only the product names** from all orders using `$unwind` and `$project`.
9. **Apply filtering using `$match`** (only Delivered orders) and then calculate revenue.
10. **Calculate total quantity and total revenue per product** (use `$unwind` and `$group`).

> **Hint:** Explore MongoDB operators: `$group`, `$match`, `$sort`, `$project`, `$unwind`, `$sum`, `$avg`, `$limit`.
> **Deliverable:** Save each aggregation pipeline as a `.json` file or screenshot of results.

Part 2: Indexing

Tasks

1. **Check indexes** on the collection.
2. **Create an index** on `customerName`. Run a query filtering by `customerName` and check performance using `explain("executionStats")`.
3. **Create a compound index** on `status` and `orderDate`. Run a query filtering by both and compare performance (before vs after).
4. **Create a text index** on `items.productName`. Perform a text search for a product.
5. **Drop an index** and observe performance difference.

Hint: Explore `db.orders.getIndexes()`, `db.orders.createIndex()`, `db.orders.dropIndex()`, and `db.orders.find(...).explain("executionStats")`.
Deliverable: Document query execution times **before and after indexing**.

Expected Deliverables

1. **A Markdown/Word/PDF file** with:
    - Dataset creation steps or sample JSON.
    - Aggregation queries and their results.
    - Indexing queries and performance comparison.
    - Observations & learnings.
2. Use your backend **GitHub repository**to add:
    - Dummy dataset script.
    - Notes on Aggregation
    - Notes on indexing.